Building datasets

In this example, we'll use LightFM's built-in `Dataset` class to build an interaction dataset from raw data. The goal is to demonstrate how to go from raw data (lists of interactions and perhaps item and user features) to `scipy.sparse` matrices that can be used to fit a LightFM model.

Getting the data

We're going to use a sample from Goodbooks-10k as our example dataset. Let's download the data first.

```
import os
import zipfile
import csv

import requests


def _download(url: str, dest_path: str):

    req = requests.get(url, stream=True)
    req.raise_for_status()

    with open(dest_path, "wb") as fd:
        for chunk in req.iter_content(chunk_size=2 ** 20):
            fd.write(chunk)


def get_data():

    ratings_url = ("http://www2.informatik.uni-freiburg.de/" "~cziegler/BX/BX-
CSV-Dump.zip")

    if not os.path.exists("data"):
        os.makedirs("data")

        _download(ratings_url, "data/data.zip")

    with zipfile.ZipFile("data/data.zip") as archive:
        return (
            csv.DictReader(
                (x.decode("utf-8", "ignore") for x in archive.open("BX-Book-
Ratings.csv")),
                delimiter=";",
            ),
            csv.DictReader(
                (x.decode("utf-8", "ignore") for x in archive.open("BX-
Books.csv")), delimiter=";"
            ),
        )


def get_ratings():

    return get_data()[0]


def get_book_features():

    return get_data()[1]
```

The data consists of book ratings and book details:

```
import json
from itertools import islice

ratings, book_features = get_data()
```

Ratings look like this:

```
for line in islice(ratings, 2):
    print(json.dumps(line, indent=4))
```


```
{
    "User-ID": "276725",
    "ISBN": "034545104X",
    "Book-Rating": "0"
}
{
    "User-ID": "276726",
    "ISBN": "0155061224",
    "Book-Rating": "5"
}
```


and book features look like this:

```
for line in islice(book_features, 1):
    print(json.dumps(line, indent=4))
```


```
{
    "ISBN": "0195153448",
    "Book-Title": "Classical Mythology",
    "Book-Author": "Mark P. O. Morford",
    "Year-Of-Publication": "2002",
    "Publisher": "Oxford University Press",
    "Image-URL-S":
"http://images.amazon.com/images/P/0195153448.01.THUMBZZZ.jpg",
    "Image-URL-M":
"http://images.amazon.com/images/P/0195153448.01.MZZZZZZZ.jpg",
    "Image-URL-L":
"http://images.amazon.com/images/P/0195153448.01.LZZZZZZZ.jpg"
}
```


Building the ID mappings

The first thing we need to do is to create a mapping between the user and item ids from our input data to indices that will be used internally by our model.

We do this because LightFM works with user and item ids that are consecutive non-negative integers. The `Dataset` class allow us to create a mapping between the IDs we use in our systems and the consecutive indices preferred by the model.

To do this, we create a dataset and call its `fit` method. The first argument is an iterable of all user ids in our data, and the second is an iterable of all item ids. In this case, we use generator expressions to lazily iterate over our data and yield user and item ids:

```
from lightfm.data import Dataset

dataset = Dataset()
dataset.fit((x['User-ID'] for x in get_ratings()),
            (x['ISBN'] for x in get_ratings()))
```

This call will assign an internal numerical id to every user and item id we pass in. These will be contiguous (from 0 to however many users and items we have), and will also determine the dimensions of the resulting LightFM model.

We can check that the mappings have been created by querying the dataset on how many users and books it knows about:

```
num_users, num_items = dataset.interactions_shape()
print('Num users: {}, num_items {}.'.format(num_users, num_items))
```

```
Num users: 105283, num_items 340553.
```

Note that if we don't have all user and items ids at once, we can repeatedly call `fit_partial` to supply additional ids. In this case, we will use this capability to add some item feature mappings:

```
dataset.fit_partial(items=(x['ISBN'] for x in get_book_features()),
                    item_features=(x['Book-Author'] for x in
get_book_features()))
```

This will create a feature for every unique author name in the dataset.

(Note that we fit some more item ids: this is to make sure our mappings are complete even if there are items in the features dataset that are not in the interactions set.)

Building the interactions matrix

Having created the mapping, we build the interaction matrix:

```
(interactions, weights) = dataset.build_interactions(((x['User-ID'], x['ISBN'])
                                                      for x in get_ratings()))

print(repr(interactions))
```

```
<105283x341762 sparse matrix of type '<class 'numpy.int32'>'
        with 1149780 stored elements in COOrdinate format>
```

This is main input into a LightFM model: it encodes the interactions betwee users and items.

Since we have item features, we can also create the item features matrix:

```
item_features = dataset.build_item_features(((x['ISBN'], [x['Book-Author']])
                                             for x in get_book_features()))
print(repr(item_features))
```

```
<341762x443805 sparse matrix of type '<class 'numpy.float32'>'
        with 613141 stored elements in Compressed Sparse Row format>
```

Building a model

This is all we need to build a LightFM model:

```
from lightfm import LightFM

model = LightFM(loss='bpr')
model.fit(interactions, item_features=item_features)
```

```
<lightfm.lightfm.LightFM at 0x7f5f0e8f7c88>
```